■ Top 50 Array Problems

Pattern-Wise Java Solutions with Explanations

12 Patterns • 50 Problems • Full Java Code • Time & Space Complexity

Pattern Legend

- Basic Array
- Two Pointer
- Sliding Window
- Prefix Sum
- Kadane's Algorithm
- Sorting Based
- Hashing
- Binary Search
- Matrix / 2D Array
- Greedy
- Stack / Deque
- Advanced / DP

Table of Contents

Basic Array

1. Find Maximum and Minimum Element
2. Second Largest Element
3. Check if Array is Sorted
4. Reverse an Array
5. Move Zeroes to End

Two Pointer

6. Two Sum (Sorted Array)
7. Remove Duplicates from Sorted Array
8. Container With Most Water
9. Valid Palindrome
10. 3Sum

Sliding Window

11. Maximum Sum Subarray of Size K
12. Longest Substring Without Repeating Characters
13. Minimum Window Substring
14. Longest Repeating Character Replacement
15. Count Subarrays with Given Sum

Prefix Sum

16. Subarray Sum Equals K
17. Range Sum Query
18. Find Pivot Index
19. Product of Array Except Self
20. Running Sum of Array

Kadane's Algorithm

21. Maximum Subarray Sum
22. Maximum Product Subarray
23. Circular Subarray Sum

Sorting Based

24. Merge Intervals
25. Insert Interval
26. Non-Overlapping Intervals
27. Sort Colors (Dutch Flag Problem)
28. Largest Number Formed from Array

Hashing

29. Two Sum (Unsorted)
30. Longest Consecutive Sequence
31. Majority Element
32. Subarray with Zero Sum
33. Find All Duplicates in Array

Binary Search

34. Binary Search
35. First and Last Position of Element
36. Search in Rotated Sorted Array
37. Find Peak Element

38. Median of Two Sorted Arrays

Matrix / 2D Array

39. Set Matrix Zeroes
40. Spiral Matrix
41. Rotate Image (90 degrees CW)
42. Search in 2D Matrix

Greedy

43. Jump Game
44. Gas Station
45. Minimum Number of Platforms

Stack / Deque

46. Next Greater Element
47. Sliding Window Maximum
48. Largest Rectangle in Histogram

Advanced / DP

49. Longest Increasing Subsequence (LIS)
50. Maximum Sum Increasing Subsequence

Basic Array

1. Find Maximum and Minimum Element

Approach

Traverse the array once, tracking max and min with two variables.

Java Code

```
public class MaxMin {
public static int[] findMaxMin(int[] arr) {
int max = arr[0], min = arr[0];
for (int i = 1; i < arr.length; i++) {
if (arr[i] > max) max = arr[i];
if (arr[i] < min) min = arr[i];
}
return new int[]{max, min}; // [max, min]
}
public static void main(String[] a) {
int[] res = findMaxMin(new int[]{3,1,9,2,7});
System.out.println("Max=" + res[0] + " Min=" + res[1]);
}
}
```

Time: O(n) **Space: O(1)**

2. Second Largest Element

Approach

Find the max, then find the largest element smaller than max.

Java Code

```
public class SecondLargest {
public static int secondLargest(int[] arr) {
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int x : arr) {
if (x > first) { second = first; first = x; }
else if (x > second && x != first) second = x;
}
return second; // returns MIN_VALUE if no second distinct
}
}
```

Time: O(n) **Space: O(1)**

3. Check if Array is Sorted

Approach

Compare each element with the next; if any pair is out of order, return false.

Java Code

```
public class IsSorted {
public static boolean isSorted(int[] arr) {
for (int i = 0; i < arr.length - 1; i++) {
if (arr[i] > arr[i + 1]) return false;
}
return true;
}
}
```

Time: O(n) **Space: O(1)**

4. Reverse an Array

Approach

Use two pointers — one at start, one at end — swap and move inward.

Java Code

```
public class ReverseArray {
public static void reverse(int[] arr) {
int l = 0, r = arr.length - 1;
while (l < r) {
int tmp = arr[l]; arr[l] = arr[r]; arr[r] = tmp;
l++; r--;
}
}
}
```

Time: O(n) **Space: O(1)**

5. Move Zeroes to End

Approach

Keep a 'write' pointer. Copy non-zero elements to front, fill rest with 0.

Java Code

```
public class MoveZeroes {
public static void moveZeroes(int[] arr) {
int pos = 0;
for (int x : arr) if (x != 0) arr[pos++] = x;
while (pos < arr.length) arr[pos++] = 0;
}
}
```

Time: O(n) **Space: O(1)**

■ Two Pointer

6. Two Sum (Sorted Array)

■ Approach

Left pointer at start, right at end. If sum < target move left++, if > target move right--.

■ Java Code

```
public class TwoSumSorted {
public static int[] twoSum(int[] nums, int target) {
int l = 0, r = nums.length - 1;
while (l < r) {
int sum = nums[l] + nums[r];
if (sum == target) return new int[]{l, r};
else if (sum < target) l++;
else r--;
}
return new int[]{-1, -1};
}
}
```

■ **Time: O(n)** ■ **Space: O(1)**

7. Remove Duplicates from Sorted Array

■ Approach

Slow pointer marks unique boundary; fast pointer scans ahead for new values.

■ Java Code

```
public class RemoveDups {
public static int removeDuplicates(int[] nums) {
if (nums.length == 0) return 0;
int slow = 0;
for (int fast = 1; fast < nums.length; fast++) {
if (nums[fast] != nums[slow]) {
slow++;
nums[slow] = nums[fast];
}
}
return slow + 1; // new length
}
}
```

■ **Time: O(n)** ■ **Space: O(1)**

8. Container With Most Water

Approach

Two pointers at ends. Area = min(height)*width. Move the shorter side inward.

Java Code

```
public class ContainerWater {
public static int maxArea(int[] height) {
int l = 0, r = height.length - 1, max = 0;
while (l < r) {
max = Math.max(max, Math.min(height[l], height[r]) * (r - l));
if (height[l] < height[r]) l++;
else r--;
}
return max;
}
}
```

Time: O(n) **Space: O(1)**

9. Valid Palindrome

Approach

Two pointers. Skip non-alphanumeric. Compare lowercased characters.

Java Code

```
public class ValidPalindrome {
public static boolean isPalindrome(String s) {
int l = 0, r = s.length() - 1;
while (l < r) {
while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
if (Character.toLowerCase(s.charAt(l)) !=
Character.toLowerCase(s.charAt(r))) return false;
l++; r--;
}
return true;
}
}
```

Time: O(n) **Space: O(1)**

10. 3Sum

Approach

Sort array. For each element, use two-pointer on the rest to find triplets summing to 0.

Java Code

```
import java.util.*;
public class ThreeSum {
public static List<List<Integer>> threeSum(int[] nums) {
```

```
Arrays.sort(nums);
List<List<Integer>> res = new ArrayList<>();
for (int i = 0; i < nums.length - 2; i++) {
if (i > 0 && nums[i] == nums[i-1]) continue; // skip dups
int l = i+1, r = nums.length-1;
while (l < r) {
int sum = nums[i] + nums[l] + nums[r];
if (sum == 0) {
res.add(Arrays.asList(nums[i], nums[l], nums[r]));
while (l < r && nums[l]==nums[l+1]) l++;
while (l < r && nums[r]==nums[r-1]) r--;
l++; r--;
} else if (sum < 0) l++;
else r--;
}
}
return res;
}
}
```

■ **Time: O(n^2)** ■ **Space: O(1)**

■ Sliding Window

11. Maximum Sum Subarray of Size K

■ Approach

Compute sum of first K elements, then slide: add next, remove first.

■ Java Code

```
public class MaxSumK {
public static int maxSum(int[] arr, int k) {
int windowSum = 0, maxSum = 0;
for (int i = 0; i < k; i++) windowSum += arr[i];
maxSum = windowSum;
for (int i = k; i < arr.length; i++) {
windowSum += arr[i] - arr[i - k];
maxSum = Math.max(maxSum, windowSum);
}
return maxSum;
}
}
```

■ **Time: O(n)** ■ **Space: O(1)**

12. Longest Substring Without Repeating Characters

■ Approach

Expand right; if duplicate found, shrink left past the previous occurrence.

■ Java Code

```
import java.util.*;
public class LongestUnique {
public static int lengthOfLongestSubstring(String s) {
Map<Character,Integer> map = new HashMap<>();
int max = 0, l = 0;
for (int r = 0; r < s.length(); r++) {
char c = s.charAt(r);
if (map.containsKey(c)) l = Math.max(l, map.get(c)+1);
map.put(c, r);
max = Math.max(max, r - l + 1);
}
return max;
}
}
```

■ **Time: O(n)** ■ **Space: O(min(n,m))**

13. Minimum Window Substring

Approach

Expand right until all chars covered; then shrink left to find minimum window.

Java Code

```
import java.util.*;
public class MinWindow {
public static String minWindow(String s, String t) {
Map<Character,Integer> need = new HashMap<>();
for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);
int have=0, req=need.size(), l=0, minLen=Integer.MAX_VALUE, start=0;
Map<Character,Integer> win = new HashMap<>();
for (int r=0; r<s.length(); r++) {
char c = s.charAt(r);
win.merge(c, 1, Integer::sum);
if (need.containsKey(c) && win.get(c).equals(need.get(c))) have++;
while (have == req) {
if (r-l+1 < minLen) { minLen=r-l+1; start=l; }
char lc = s.charAt(l++);
win.merge(lc,-1,Integer::sum);
if (need.containsKey(lc) && win.get(lc)<need.get(lc)) have--;
}
}
return minLen==Integer.MAX_VALUE ? "" : s.substring(start,start+minLen);
}
}
```

Time: O(n+m) **Space: O(m)**

14. Longest Repeating Character Replacement

Approach

Track the max frequency char in window. Replace others if (window size - maxFreq) <= k.

Java Code

```
public class CharReplacement {
public static int characterReplacement(String s, int k) {
int[] count = new int[26];
int max = 0, l = 0, maxFreq = 0;
for (int r = 0; r < s.length(); r++) {
count[s.charAt(r)-'A']++;
maxFreq = Math.max(maxFreq, count[s.charAt(r)-'A']);
if ((r - l + 1) - maxFreq > k) {
count[s.charAt(l)-'A']--;
l++;
}
max = Math.max(max, r - l + 1);
}
return max;
}
```

```
}
```

■ Time: O(n) ■ Space: O(1)

15. Count Subarrays with Given Sum

■ Approach

Use prefix sum + HashMap. Count subarrays where prefixSum[j]-prefixSum[i]==k.

■ Java Code

```
import java.util.*;
public class SubarraySum {
public static int subarraySum(int[] nums, int k) {
Map<Integer,Integer> map = new HashMap<>();
map.put(0, 1);
int sum = 0, count = 0;
for (int n : nums) {
sum += n;
count += map.getOrDefault(sum - k, 0);
map.merge(sum, 1, Integer::sum);
}
return count;
}
}
```

■ Time: O(n) ■ Space: O(n)

■ Prefix Sum

16. Subarray Sum Equals K

■ Approach

Running prefix sum; store frequencies in HashMap. For each sum, check if (sum-k) exists.

■ Java Code

```
import java.util.*;
public class SubarraySumK {
public static int subarraySum(int[] nums, int k) {
Map<Integer,Integer> map = new HashMap<>();
map.put(0, 1);
int sum = 0, ans = 0;
for (int n : nums) {
sum += n;
ans += map.getOrDefault(sum - k, 0);
map.merge(sum, 1, Integer::sum);
}
return ans;
}
}
```

■ **Time: O(n)** ■ **Space: O(n)**

17. Range Sum Query

■ Approach

Precompute prefix[] where prefix[i] = sum of first i elements. Range sum = prefix[r]-prefix[l-1].

■ Java Code

```
public class RangeSum {
private int[] prefix;
public RangeSum(int[] nums) {
prefix = new int[nums.length + 1];
for (int i = 0; i < nums.length; i++)
prefix[i+1] = prefix[i] + nums[i];
}
public int sumRange(int l, int r) {
return prefix[r+1] - prefix[l]; // O(1) query
}
}
```

■ **Time: O(n) build, O(1) query** ■ **Space: O(n)**

18. Find Pivot Index

Approach

Pivot where leftSum == rightSum. Use total sum; track leftSum as you traverse.

Java Code

```
public class PivotIndex {
public static int pivotIndex(int[] nums) {
int total = 0;
for (int n : nums) total += n;
int left = 0;
for (int i = 0; i < nums.length; i++) {
if (left == total - left - nums[i]) return i;
left += nums[i];
}
return -1;
}
}
```

Time: O(n) **Space: O(1)**

19. Product of Array Except Self

Approach

Build left-product and right-product arrays, combine them without division.

Java Code

```
public class ProductExceptSelf {
public static int[] productExceptSelf(int[] nums) {
int n = nums.length;
int[] res = new int[n];
res[0] = 1;
for (int i = 1; i < n; i++) res[i] = res[i-1] * nums[i-1];
int right = 1;
for (int i = n-1; i >= 0; i--) {
res[i] *= right;
right *= nums[i];
}
return res;
}
}
```

Time: O(n) **Space: O(1) extra**

20. Running Sum of Array

Approach

Each element becomes the sum of itself and all previous elements.

Java Code

```
public class RunningSum {
```

```
public static int[] runningSum(int[] nums) {
for (int i = 1; i < nums.length; i++)
nums[i] += nums[i-1];
return nums;
}
}
```

■ **Time: O(n)** ■ **Space: O(1)**

■ Kadane's Algorithm

21. Maximum Subarray Sum

■ Approach

At each step: curSum = max(num, curSum+num). Track global max.

■ Java Code

```
public class MaxSubarray {
public static int maxSubArray(int[] nums) {
int cur = nums[0], max = nums[0];
for (int i = 1; i < nums.length; i++) {
cur = Math.max(nums[i], cur + nums[i]);
max = Math.max(max, cur);
}
return max;
}
}
```

■ **Time: O(n)** ■ **Space: O(1)**

22. Maximum Product Subarray

■ Approach

Track both max and min products (negatives can flip). Update at each step.

■ Java Code

```
public class MaxProduct {
public static int maxProduct(int[] nums) {
int max = nums[0], min = nums[0], res = nums[0];
for (int i = 1; i < nums.length; i++) {
int tmp = max;
max = Math.max(nums[i], Math.max(max*nums[i], min*nums[i]));
min = Math.min(nums[i], Math.min(tmp*nums[i], min*nums[i]));
res = Math.max(res, max);
}
return res;
}
}
```

■ **Time: O(n)** ■ **Space: O(1)**

23. Circular Subarray Sum

■ Approach

Answer is max of: normal Kadane, OR total_sum - min_subarray (wrapping case).

Java Code

```
public class CircularSubarray {
public static int maxSubarraySumCircular(int[] nums) {
int total=0, maxSum=nums[0], curMax=0, minSum=nums[0], curMin=0;
for (int n : nums) {
curMax = Math.max(curMax + n, n); maxSum = Math.max(maxSum, curMax);
curMin = Math.min(curMin + n, n); minSum = Math.min(minSum, curMin);
total += n;
}
return maxSum > 0 ? Math.max(maxSum, total - minSum) : maxSum;
}
}
```

Time: O(n) **Space: O(1)**

Sorting Based

24. Merge Intervals

Approach

Sort by start. For each interval: if overlap with last, extend end; else add new.

Java Code

```
import java.util.*;
public class MergeIntervals {
public static int[][] merge(int[][] intervals) {
Arrays.sort(intervals, (a,b) -> a[0]-b[0]);
List<int[]> res = new ArrayList<>();
for (int[] iv : intervals) {
if (res.isEmpty() || res.get(res.size()-1)[1] < iv[0])
res.add(iv);
else
res.get(res.size()-1)[1] = Math.max(res.get(res.size()-1)[1], iv[1]);
}
return res.toArray(new int[0][]);
}
}
```

Time: O(n log n) **Space: O(n)**

25. Insert Interval

Approach

Add new interval, merge all overlapping intervals in a single pass.

Java Code

```
import java.util.*;
public class InsertInterval {
public static int[][] insert(int[][] intervals, int[] newInterval) {
List<int[]> res = new ArrayList<>();
int i = 0, n = intervals.length;
while (i < n && intervals[i][1] < newInterval[0]) res.add(intervals[i++]);
while (i < n && intervals[i][0] <= newInterval[1]) {
newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
i++;
}
res.add(newInterval);
while (i < n) res.add(intervals[i++]);
return res.toArray(new int[0][]);
}
}
```

■ Time: O(n) ■ Space: O(n)

26. Non-Overlapping Intervals

■ Approach

Sort by end time. Greedily keep intervals with earliest end, count removals.

■ Java Code

```
import java.util.*;
public class NonOverlapping {
public static int eraseOverlapIntervals(int[][] intervals) {
Arrays.sort(intervals, (a,b) -> a[1]-b[1]);
int count = 0, end = Integer.MIN_VALUE;
for (int[] iv : intervals) {
if (iv[0] >= end) end = iv[1];
else count++;
}
return count;
}
}
```

■ Time: O(n log n) ■ Space: O(1)

27. Sort Colors (Dutch Flag Problem)

■ Approach

Three pointers: lo=0, mid=0, hi=n-1. Swap 0s left, 2s right, leave 1s in middle.

■ Java Code

```
public class SortColors {
public static void sortColors(int[] nums) {
int lo = 0, mid = 0, hi = nums.length - 1;
while (mid <= hi) {
if (nums[mid] == 0) { swap(nums, lo++, mid++); }
else if (nums[mid] == 1) { mid++; }
else { swap(nums, mid, hi--); }
}
}
static void swap(int[] a, int i, int j) {
int t = a[i]; a[i] = a[j]; a[j] = t; }
}
```

■ Time: O(n) ■ Space: O(1)

28. Largest Number Formed from Array

■ Approach

Custom sort: compare concatenations (b+a) vs (a+b). Join sorted array.

Java Code

```
import java.util.*;
public class LargestNumber {
public static String largestNumber(int[] nums) {
String[] strs = new String[nums.length];
for (int i = 0; i < nums.length; i++) strs[i] = String.valueOf(nums[i]);
Arrays.sort(strs, (a,b) -> (b+a).compareTo(a+b));
if (strs[0].equals("0")) return "0";
StringBuilder sb = new StringBuilder();
for (String s : strs) sb.append(s);
return sb.toString();
}
}
```

Time: O(n log n)

Space: O(n)

■ Hashing

29. Two Sum (Unsorted)

■ Approach

Store each element in HashMap with index. For each num, check if (target-num) exists.

■ Java Code

```
import java.util.*;
public class TwoSum {
public static int[] twoSum(int[] nums, int target) {
Map<Integer,Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
int comp = target - nums[i];
if (map.containsKey(comp)) return new int[]{map.get(comp), i};
map.put(nums[i], i);
}
return new int[]{};
}
}
```

■ **Time: O(n)** ■ **Space: O(n)**

30. Longest Consecutive Sequence

■ Approach

Put all in HashSet. For each start (no n-1 in set), count consecutive streak.

■ Java Code

```
import java.util.*;
public class LongestConsecutive {
public static int longestConsecutive(int[] nums) {
Set<Integer> set = new HashSet<>();
for (int n : nums) set.add(n);
int best = 0;
for (int n : set) {
if (!set.contains(n-1)) {
int cur = n, len = 1;
while (set.contains(++cur)) len++;
best = Math.max(best, len);
}
}
return best;
}
}
```

■ **Time: O(n)** ■ **Space: O(n)**

31. Majority Element

Approach

Boyer-Moore Voting: maintain candidate and count. Count > 0 means candidate survives.

Java Code

```
public class MajorityElement {
public static int majorityElement(int[] nums) {
int candidate = nums[0], count = 1;
for (int i = 1; i < nums.length; i++) {
if (count == 0) { candidate = nums[i]; count = 1; }
else if (nums[i] == candidate) count++;
else count--;
}
return candidate; // guaranteed to exist
}
}
```

Time: O(n) **Space: O(1)**

32. Subarray with Zero Sum

Approach

If prefix sum repeats, subarray between those indices sums to 0.

Java Code

```
import java.util.*;
public class ZeroSumSubarray {
public static boolean hasZeroSum(int[] arr) {
Set<Integer> set = new HashSet<>();
set.add(0);
int sum = 0;
for (int n : arr) {
sum += n;
if (!set.add(sum)) return true; // sum seen before
}
return false;
}
}
```

Time: O(n) **Space: O(n)**

33. Find All Duplicates in Array

Approach

For each number n, negate arr[|n|-1]. If already negative, it's a duplicate.

Java Code

```
import java.util.*;
public class FindDuplicates {
public static List<Integer> findDuplicates(int[] nums) {
List<Integer> res = new ArrayList<>();
for (int n : nums) {
int idx = Math.abs(n) - 1;
if (nums[idx] < 0) res.add(Math.abs(n));
else nums[idx] = -nums[idx];
}
return res;
}
}
```

■ **Time: O(n)**

■ **Space: O(1) extra**

Binary Search

34. Binary Search

Approach

Classic: compare mid with target, eliminate half each iteration.

Java Code

```
public class BinarySearch {
public static int search(int[] nums, int target) {
int l = 0, r = nums.length - 1;
while (l <= r) {
int mid = l + (r - l) / 2;
if (nums[mid] == target) return mid;
else if (nums[mid] < target) l = mid + 1;
else r = mid - 1;
}
return -1;
}
}
```

Time: O(log n) **Space: O(1)**

35. First and Last Position of Element

Approach

Two binary searches: one for leftmost occurrence, one for rightmost.

Java Code

```
public class FirstLastPosition {
public static int[] searchRange(int[] nums, int target) {
return new int[]{find(nums,target,true), find(nums,target,false)};
}
static int find(int[] nums, int t, boolean first) {
int l=0, r=nums.length-1, ans=-1;
while (l<=r) {
int mid=l+(r-l)/2;
if (nums[mid]==t) {
ans=mid;
if (first) r=mid-1; else l=mid+1;
} else if (nums[mid]<t) l=mid+1;
else r=mid-1;
}
return ans;
}
}
```

■ Time: O(log n) ■ Space: O(1)

36. Search in Rotated Sorted Array

■ Approach

Determine which half is sorted; check if target is in that half, else search other.

■ Java Code

```
public class RotatedSearch {
public static int search(int[] nums, int target) {
int l = 0, r = nums.length - 1;
while (l <= r) {
int mid = l + (r-l)/2;
if (nums[mid] == target) return mid;
if (nums[l] <= nums[mid]) { // left half sorted
if (nums[l] <= target && target < nums[mid]) r = mid-1;
else l = mid+1;
} else { // right half sorted
if (nums[mid] < target && target <= nums[r]) l = mid+1;
else r = mid-1;
}
}
return -1;
}
}
```

■ Time: O(log n) ■ Space: O(1)

37. Find Peak Element

■ Approach

If mid > mid+1, peak is on left side. Otherwise on right side.

■ Java Code

```
public class FindPeak {
public static int findPeakElement(int[] nums) {
int l = 0, r = nums.length - 1;
while (l < r) {
int mid = l + (r-l)/2;
if (nums[mid] > nums[mid+1]) r = mid;
else l = mid + 1;
}
return l; // index of peak
}
}
```

■ Time: O(log n) ■ Space: O(1)

38. Median of Two Sorted Arrays

Approach

Binary search on shorter array partitions. Ensure left halves have correct sizes.

Java Code

```
public class MedianSortedArrays {
public static double findMedianSortedArrays(int[] A, int[] B) {
if (A.length > B.length) return findMedianSortedArrays(B, A);
int m=A.length, n=B.length, lo=0, hi=m;
while (lo<=hi) {
int i=lo+(hi-lo)/2, j=(m+n+1)/2-i;
int aL=i>0?A[i-1]:Integer.MIN_VALUE;
int aR=i<m?A[i]:Integer.MAX_VALUE;
int bL=j>0?B[j-1]:Integer.MIN_VALUE;
int bR=j<n?B[j]:Integer.MAX_VALUE;
if (aL<=bR && bL<=aR) {
if ((m+n)%2==1) return Math.max(aL,bL);
return (Math.max(aL,bL)+Math.min(aR,bR))/2.0;
} else if (aL>bR) hi=i-1;
else lo=i+1;
}
return 0;
}
}
```

Time: O(log(min(m,n))) **Space: O(1)**

■ Matrix / 2D Array

39. Set Matrix Zeroes

■ Approach

Use first row/col as markers. First scan marks, second scan zeros, handle first row/col last.

■ Java Code

```
public class SetZeroes {
public static void setZeroes(int[][] matrix) {
int m=matrix.length, n=matrix[0].length;
boolean firstRow=false, firstCol=false;
for(int j=0;j<n;j++) if(matrix[0][j]==0) firstRow=true;
for(int i=0;i<m;i++) if(matrix[i][0]==0) firstCol=true;
for(int i=1;i<m;i++) for(int j=1;j<n;j++)
if(matrix[i][j]==0){matrix[i][0]=0;matrix[0][j]=0;}
for(int i=1;i<m;i++) for(int j=1;j<n;j++)
if(matrix[i][0]==0||matrix[0][j]==0) matrix[i][j]=0;
if(firstRow) for(int j=0;j<n;j++) matrix[0][j]=0;
if(firstCol) for(int i=0;i<m;i++) matrix[i][0]=0;
}
}
```

■ **Time: O(m*n)** ■ **Space: O(1)**

40. Spiral Matrix

■ Approach

Maintain top/bottom/left/right boundaries. Traverse right□down□left□up, shrink bounds.

■ Java Code

```
import java.util.*;
public class SpiralMatrix {
public static List<Integer> spiralOrder(int[][] matrix) {
List<Integer> res = new ArrayList<>();
int t=0,b=matrix.length-1,l=0,r=matrix[0].length-1;
while(t<=b && l<=r) {
for(int j=l;j<=r;j++) res.add(matrix[t][j]); t++;
for(int i=t;i<=b;i++) res.add(matrix[i][r]); r--;
if(t<=b){for(int j=r;j>=l;j--) res.add(matrix[b][j]); b--;}
if(l<=r){for(int i=b;i>=t;i--) res.add(matrix[i][l]); l++;}
}
return res;
}
}
```

■ **Time: O(m*n)** ■ **Space: O(1)**

41. Rotate Image (90 degrees CW)

Approach

Transpose the matrix (swap [i][j] and [j][i]), then reverse each row.

Java Code

```
public class RotateImage {
public static void rotate(int[][] matrix) {
int n = matrix.length;
// Step 1: Transpose
for(int i=0;i<n;i++) for(int j=i+1;j<n;j++) {
int tmp=matrix[i][j]; matrix[i][j]=matrix[j][i]; matrix[j][i]=tmp;
}
// Step 2: Reverse each row
for(int[] row : matrix) {
int l=0, r=n-1;
while(l<r){int t=row[l];row[l++]=row[r];row[r--]=t;}
}
}
}
```

Time: O(n^2) **Space: O(1)**

42. Search in 2D Matrix

Approach

Start from top-right corner. If target < current go left; if target > go down.

Java Code

```
public class Search2DMatrix {
public static boolean searchMatrix(int[][] matrix, int target) {
int r=0, c=matrix[0].length-1;
while(r<matrix.length && c>=0) {
if(matrix[r][c]==target) return true;
else if(matrix[r][c]>target) c--;
else r++;
}
return false;
}
}
```

Time: O(m+n) **Space: O(1)**

Greedy

43. Jump Game

Approach

Track the farthest index reachable. If current index > farthest, can't proceed.

Java Code

```
public class JumpGame {
public static boolean canJump(int[] nums) {
int farthest = 0;
for (int i = 0; i < nums.length; i++) {
if (i > farthest) return false;
farthest = Math.max(farthest, i + nums[i]);
}
return true;
}
}
```

Time: O(n) **Space: O(1)**

44. Gas Station

Approach

If total gas >= total cost, solution exists. Start from last deficit point.

Java Code

```
public class GasStation {
public static int canCompleteCircuit(int[] gas, int[] cost) {
int total=0, tank=0, start=0;
for(int i=0;i<gas.length;i++) {
total += gas[i]-cost[i];
tank += gas[i]-cost[i];
if(tank < 0) { start=i+1; tank=0; }
}
return total>=0 ? start : -1;
}
}
```

Time: O(n) **Space: O(1)**

45. Minimum Number of Platforms

Approach

Sort arrivals and departures. Use two pointers; count simultaneous trains.

Java Code

```
import java.util.*;
public class MinPlatforms {
public static int findPlatform(int[] arr, int[] dep) {
Arrays.sort(arr); Arrays.sort(dep);
int plat=1, maxPlat=1, i=1, j=0;
while(i<arr.length && j<dep.length) {
if(arr[i]<=dep[j]) { plat++; i++; }
else { plat--; j++; }
maxPlat = Math.max(maxPlat, plat);
}
return maxPlat;
}
}
```

- **Time: O(n log n)**
- **Space: O(1)**

Stack / Deque

46. Next Greater Element

Approach

Use a monotonic stack. For each element, pop while stack top is smaller.

Java Code

```
import java.util.*;
public class NextGreater {
public static int[] nextGreaterElement(int[] nums) {
int n=nums.length;
int[] res = new int[n];
Arrays.fill(res,-1);
Deque<Integer> stack = new ArrayDeque<>(); // stores indices
for(int i=0;i<n;i++) {
while(!stack.isEmpty() && nums[stack.peek()]<nums[i])
res[stack.pop()] = nums[i];
stack.push(i);
}
return res;
}
}
```

Time: O(n) **Space: O(n)**

47. Sliding Window Maximum

Approach

Use a monotone decreasing deque of indices. Front is always the window max.

Java Code

```
import java.util.*;
public class SlidingWindowMax {
public static int[] maxSlidingWindow(int[] nums, int k) {
int n=nums.length;
int[] res = new int[n-k+1];
Deque<Integer> dq = new ArrayDeque<>(); // stores indices
for(int i=0;i<n;i++) {
if(!dq.isEmpty() && dq.peekFirst()<i-k+1) dq.pollFirst();
while(!dq.isEmpty() && nums[dq.peekLast()]<nums[i]) dq.pollLast();
dq.offerLast(i);
if(i>=k-1) res[i-k+1]=nums[dq.peekFirst()];
}
return res;
}
}
```

■ Time: O(n) ■ Space: O(k)

48. Largest Rectangle in Histogram

■ Approach

Monotone increasing stack. On pop, compute area with popped bar as the shortest.

■ Java Code

```
import java.util.*;
public class LargestRectangle {
public static int largestRectangleArea(int[] heights) {
Deque<Integer> stack = new ArrayDeque<>();
int max=0, n=heights.length;
for(int i=0;i<=n;i++) {
int h = (i==n)?0:heights[i];
while(!stack.isEmpty() && heights[stack.peek()]>h) {
int height=heights[stack.pop()];
int width=stack.isEmpty()?i:i-stack.peek()-1;
max=Math.max(max,height*width);
}
stack.push(i);
}
return max;
}
}
```

■ Time: O(n) ■ Space: O(n)

■ Advanced / DP

49. Longest Increasing Subsequence (LIS)

■ Approach

dp[i] = length of LIS ending at index i. Check all j < i where nums[j] < nums[i].

■ Java Code

```
public class LIS {
public static int lengthOfLIS(int[] nums) {
int n=nums.length, max=1;
int[] dp = new int[n];
java.util.Arrays.fill(dp, 1);
for(int i=1;i<n;i++) {
for(int j=0;j<i;j++) {
if(nums[j]<nums[i])
dp[i]=Math.max(dp[i], dp[j]+1);
}
max=Math.max(max,dp[i]);
}
return max;
}
// O(n log n) with patience sorting / binary search also possible
}
```

■ **Time: O(n^2)** ■ **Space: O(n)**

50. Maximum Sum Increasing Subsequence

■ Approach

dp[i] = max sum of increasing subsequence ending at i. Similar to LIS but tracking sum.

■ Java Code

```
public class MaxSumIS {
public static int maxSumIS(int[] arr) {
int n=arr.length, max=0;
int[] dp = arr.clone(); // base: each element is its own subsequence
for(int i=1;i<n;i++) {
for(int j=0;j<i;j++) {
if(arr[j]<arr[i] && dp[j]+arr[i]>dp[i])
dp[i]=dp[j]+arr[i];
}
max=Math.max(max,dp[i]);
}
return max;
}
}
```

■ Time: O(n^2) ■ Space: O(n)